

Quantum Boltzmann Machines and QGANs

QML

Contents

1	Overview	3
2	Classical Boltzmann Machines Recap	3
2.1	Energy-Based Models	3
2.2	Restricted Boltzmann Machines (RBMs)	4
2.3	Contrastive Divergence Training	4
3	Classical GANs Recap	5
3.1	The GAN Framework	5
3.2	Training Dynamics	5
3.3	Wasserstein Distance	5
4	Quantum Boltzmann Machines	6
4.1	From Classical to Quantum Energy	6
4.2	Quantum Thermal States	6
4.3	QBM Training	7
4.4	Variational QBM (Zoufal et al.)	7
5	Quantum GANs — The Main Framework	8
5.1	QGAN Architecture	8
5.2	QGAN Generator Circuit	8
5.3	Noise Input for QGANs	9
6	Training QGANs	9
6.1	The Quantum Minimax Game	9
6.2	Alternating Optimization	10
6.3	Loss Functions for QGANs	10
7	Zoufal et al. 2019 — The Foundational QGAN Paper	10
7.1	Key Contributions	11
7.2	The Loading Problem	11
7.3	Architecture Details	11
7.4	Results	11

8	Born Machines	12
8.1	The Born Rule as a Generative Model	12
8.2	Training Born Machines	12
8.3	Quantum Circuit Born Machine (QCBM)	13
9	Applications	13
9.1	Financial Data Generation	13
9.2	Drug Molecule Generation	14
9.3	Anomaly Detection	14
10	QGAN Training Loop — Visual	14
11	Born Machine Sampling Concept	15
12	Mode Collapse and Training Challenges	15
12.1	Mode Collapse	15
12.2	Shot Noise	15
12.3	Hardware Noise	16
12.4	Barren Plateaus (Again)	16
13	Comparison: QBM vs QGAN vs Born Machine	16
14	Advanced Topics	17
14.1	Fully Quantum GANs	17
14.2	Conditional QGANs	17
14.3	Quantum Wasserstein GAN	18
15	Practical Considerations	18
15.1	How Many Qubits Do U Need?	18
15.2	Circuit Depth vs Expressivity	18
15.3	Comparison with Classical Loading	19
16	Summary	19
17	Further Reading	19

1 Overview

so far in this course we've mostly focused on discriminative models — classifiers, regressors, stuff that maps inputs to outputs. but some of the most exciting ML is generative: learning to produce new data that looks like ur training set. think GANs generating faces, or Boltzmann machines learning probability distributions.

this lecture we dive into the quantum versions of these generative models. specifically: quantum Boltzmann machines (QBM), quantum GANs (QGANs), and Born machines. these r genuinely cool bcs they exploit quantum mechanics in a deep way — quantum superposition gives u access to exponentially large probability distributions that would be hard to represent classically.

the roadmap: first well recap the classical versions (Boltzmann machines and GANs), then quantize them, then discuss the key papers (zoufal et al. 2019 is the big one for QGANs), and finally look at applications and challenges.

fair warning: generative quantum models r still pretty experimental. training is hard, hardware is noisy, and the advantage question is open. but the ideas r beautiful, and some of the applications (loading distributions for quantum Monte Carlo, drug molecule generation) r genuinely promising.

2 Classical Boltzmann Machines Recap

before we quantize anything, lets make sure the classical foundations r solid.

2.1 Energy-Based Models

Definition 2.1 (Energy-Based Model). an energy-based model assigns an energy $E(\mathbf{x})$ to each configuration $\mathbf{x}$ of binary variables. the probability of a configuration is given by the Boltzmann distribution:

$$p(\mathbf{x}) = \frac{e^{-E(\mathbf{x})}}{Z}, \quad Z = \sum_{\mathbf{x}} e^{-E(\mathbf{x})}$$

where Z is the partition function (normalization constant).

Intuition. low energy = high probability. the model learns to assign low energy to configurations that look like training data, and high energy to everything else. its like a landscape where the training data sits in valleys.

for a general Boltzmann machine with visible units $\mathbf{v}$ and hidden units $\mathbf{h}$, the energy is:

$$E(\mathbf{v}, \mathbf{h}) = -\sum_i a_i v_i - \sum_j b_j h_j - \sum_{i,j} W_{ij} v_i h_j - \sum_{i < i'} L_{ii'} v_i v_{i'} - \sum_{j < j'} M_{jj'} h_j h_{j'}$$

where a_i, b_j r biases, W_{ij} r visible-hidden weights, and L, M r intra-layer connections.

the marginal probability of the visible units is:

$$p(\mathbf{v}) = \frac{1}{Z} \sum_{\mathbf{h}} e^{-E(\mathbf{v}, \mathbf{h})}$$

this is wt we actually care about — the distribution over things we can observe.

2.2 Restricted Boltzmann Machines (RBMs)

Definition 2.2 (RBM). a restricted Boltzmann machine is a Boltzmann machine where there are no intra-layer connections ($L = 0, M = 0$). visible units only connect to hidden units.

the restriction is crucial bcs it makes the conditional distributions tractable:

$$p(h_j = 1|\mathbf{v}) = \sigma \left(b_j + \sum_i W_{ij} v_i \right) \quad (1)$$

$$p(v_i = 1|\mathbf{h}) = \sigma \left(a_i + \sum_j W_{ij} h_j \right) \quad (2)$$

where $\sigma(x) = 1/(1 + e^{-x})$ is the sigmoid function.

the energy for an RBM simplifies to:

$$E(\mathbf{v}, \mathbf{h}) = -\mathbf{a}^T \mathbf{v} - \mathbf{b}^T \mathbf{h} - \mathbf{v}^T \mathbf{W} \mathbf{h}$$

Key Point. the RBM structure (bipartite graph, no intra-layer connections) is what makes training feasible. without it, computing the gradient requires sampling from a distribution that has no nice factored form.

2.3 Contrastive Divergence Training

training a Boltzmann machine means maximizing the log-likelihood:

$$\mathcal{L} = \frac{1}{N} \sum_{n=1}^N \log p(\mathbf{v}^{(n)})$$

the gradient has two terms (the “positive” and “negative” phases):

$$\frac{\partial \mathcal{L}}{\partial W_{ij}} = \langle v_i h_j \rangle_{\text{data}} - \langle v_i h_j \rangle_{\text{model}}$$

the positive phase $\langle \cdot \rangle_{\text{data}}$ is easy to compute bcs of the RBM structure: just clamp the visible units to training data and sample hidden units.

the negative phase $\langle \cdot \rangle_{\text{model}}$ requires sampling from the model distribution, which is hard. contrastive divergence (CD- k) approximates this by running only k steps of Gibbs sampling:

1. start from a training example $\mathbf{v}^{(0)}$
2. sample $\mathbf{h}^{(0)} \sim p(\mathbf{h}|\mathbf{v}^{(0)})$
3. sample $\mathbf{v}^{(1)} \sim p(\mathbf{v}|\mathbf{h}^{(0)})$
4. repeat k times
5. approximate: $\langle v_i h_j \rangle_{\text{model}} \approx v_i^{(k)} \cdot p(h_j = 1|\mathbf{v}^{(k)})$

in practice, CD-1 (just one step) works surprisingly well. hinton showed this in 2002 and it made RBMs practical.

3 Classical GANs Recap

3.1 The GAN Framework

GANs (goodfellow et al. 2014) r a completely different approach to generative modeling.

Definition 3.1 (GAN). a generative adversarial network consists of two networks:

- generator G : takes random noise $\mathbf{z} \sim p_z$ and produces fake data $G(\mathbf{z})$
- discriminator D : takes data (real or fake) and outputs probability that its real

they r trained adversarially: G tries to fool D , and D tries to catch fakes.

the minimax objective:

$$\min_G \max_D V(G, D) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z} [\log(1 - D(G(\mathbf{z})))]$$

Theorem 3.1 (Nash Equilibrium). if G and D have unlimited capacity, the unique global optimum of the minimax game is:

1. the generator perfectly reproduces the data distribution: $p_G = p_{\text{data}}$
2. the discriminator outputs $1/2$ everywhere (cant tell real from fake)

at this equilibrium, $V(G^*, D^*) = -\log 4$.

Intuition. think of it as a counterfeiter (G) vs a detective (D). the counterfeiter keeps improving until the detective literally cant tell the difference. at Nash equilibrium, the counterfeits r perfect and the detective is just guessing.

3.2 Training Dynamics

in practice, we alternate gradient steps:

1. discriminator step: fix G , update D to maximize $V(G, D)$
2. generator step: fix D , update G to minimize $V(G, D)$

the famous problem with GANs: training is unstable. u can get:

- mode collapse: G produces only a few types of outputs instead of the full diversity
- oscillations: G and D chase each other without converging
- vanishing gradients: if D is too good, G gets no useful gradient signal

these problems dont go away when we go quantum — tbh they sometimes get worse.

3.3 Wasserstein Distance

the Wasserstein GAN (WGAN) replaces the JS divergence with the Wasserstein-1 distance:

$$W(p_{\text{data}}, p_G) = \sup_{\|f\|_L \leq 1} [\mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [f(\mathbf{x})] - \mathbb{E}_{\mathbf{x} \sim p_G} [f(\mathbf{x})]]$$

this gives smoother gradients and more stable training. the Wasserstein distance is particularly relevant for QGANs bcs it helps with the shot noise problem (more on this later).

4 Quantum Boltzmann Machines

now lets quantize. the key idea is simple: replace the classical energy function with a quantum Hamiltonian.

4.1 From Classical to Quantum Energy

Definition 4.1 (Quantum Boltzmann Machine). a quantum Boltzmann machine (QBM) is defined by a Hamiltonian H acting on qubits. the model state is the thermal (Gibbs) state:

$$\rho = \frac{e^{-\beta H}}{Z}, \quad Z = \text{Tr}(e^{-\beta H})$$

where β is the inverse temperature.

the crucial difference from classical: the Hamiltonian can contain non-commuting terms. a typical QBM Hamiltonian is:

$$H = -\sum_i h_i Z_i - \sum_{i < j} J_{ij} Z_i Z_j - \sum_i \Gamma_i X_i$$

the first two terms r just the classical Ising model (diagonal in the computational basis). the third term — the transverse field — is wt makes it quantum.

Key Point. the transverse field terms $\Gamma_i X_i$ create quantum superpositions and entanglement in the thermal state. this means ρ can represent correlations that no classical Boltzmann machine can efficiently represent. this is where the potential quantum advantage lives.

4.2 Quantum Thermal States

lets be precise about wt the thermal state looks like. if we write H in its eigenbasis:

$$H = \sum_k E_k |E_k\rangle \langle E_k|$$

then:

$$\rho = \frac{1}{Z} \sum_k e^{-\beta E_k} |E_k\rangle \langle E_k|, \quad Z = \sum_k e^{-\beta E_k}$$

for a purely classical Hamiltonian (only Z_i terms), the eigenstates r computational basis states and ρ is a diagonal density matrix — just a classical probability distribution.

but when u add transverse field terms, the eigenstates r superpositions of computational basis states, and ρ has off-diagonal elements (quantum coherences):

Example 4.1. single qubit with $H = -hZ - \Gamma X$:

$$H = -\begin{pmatrix} h & \Gamma \\ \Gamma & -h \end{pmatrix}$$

eigenvalues: $E_{\pm} = \mp \sqrt{h^2 + \Gamma^2}$

the thermal state has coherences between $|0\rangle$ and $|1\rangle$ — its not just a mixture of up and down, but a genuine quantum state. the off-diagonal elements of ρ in the computational basis r nonzero whenever $\Gamma \neq 0$.

4.3 QBM Training

to train a QBM, we want to maximize the quantum log-likelihood. if our training data consists of states $\{\sigma_n\}$, the log-likelihood is:

$$\mathcal{L} = \frac{1}{N} \sum_{n=1}^N \text{Tr}(\sigma_n \log \rho)$$

equivalently, we can minimize the quantum relative entropy:

$$S(\sigma \parallel \rho) = \text{Tr}(\sigma \log \sigma) - \text{Tr}(\sigma \log \rho)$$

Definition 4.2 (QBM Gradient). the gradient of the log-likelihood with respect to a parameter θ_k (appearing in $H = \sum_k \theta_k O_k$) is:

$$\frac{\partial \mathcal{L}}{\partial \theta_k} = \beta (\langle O_k \rangle_\rho - \langle O_k \rangle_\sigma)$$

where $\langle O_k \rangle_\rho = \text{Tr}(O_k \rho)$ is the expectation in the model state and $\langle O_k \rangle_\sigma = \text{Tr}(O_k \sigma)$ is the expectation in the data state.

this looks just like the classical case (positive phase minus negative phase), but theres a subtlety: when terms in H dont commute, the gradient formula involves the Kubo-Mori inner product and things get more complicated. the formula above is exact only when all terms commute.

for the general non-commuting case, the exact gradient involves the integral:

$$\frac{\partial \mathcal{L}}{\partial \theta_k} = \beta \int_0^1 \text{Tr} \left(e^{-s\beta H} O_k e^{-(1-s)\beta H} \rho \right) ds - \beta \langle O_k \rangle_\sigma$$

which is hard to compute. this is why approximate methods r important.

4.4 Variational QBM (Zoufal et al.)

Key Point. the big practical problem with QBMs: preparing the thermal state $e^{-\beta H}/Z$ on a quantum computer is hard. u cant just “cool down” a quantum processor to the right temperature (well, u can on quantum annealers, but not on gate-based hardware).

the variational approach: use a parameterized quantum circuit (PQC) to approximate the thermal state.

Definition 4.3 (Variational QBM). a variational QBM uses a PQC $U(\phi)$ to prepare an approximate thermal state. starting from a reference state $|0\rangle^{\otimes n}$:

$$\rho(\phi) = \text{Tr}_A \left[U(\phi) |0\rangle \langle 0|^{\otimes (n+m)} U(\phi)^\dagger \right]$$

where we trace out m ancilla qubits to create a mixed state on n system qubits.

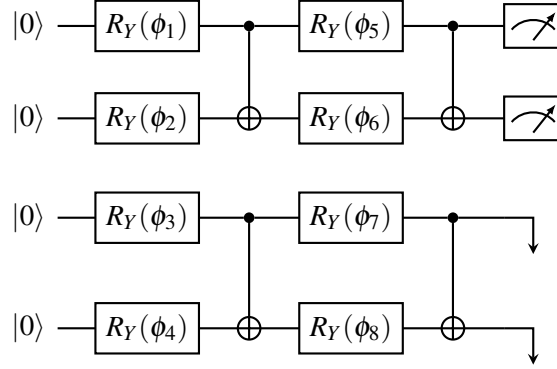
the training objective is to minimize:

$$\mathcal{C}(\phi) = S(\sigma_{\text{data}} \parallel \rho(\phi)) = -S(\sigma_{\text{data}}) - \text{Tr}(\sigma_{\text{data}} \log \rho(\phi))$$

since $S(\sigma_{\text{data}})$ is constant, we just need to maximize $\text{Tr}(\sigma_{\text{data}} \log \rho(\phi))$.

this is still hard bcs computing $\log \rho$ is expensive. in practice, people use approximations like truncated Taylor series or variational bounds.

the circuit for a variational QBM typically looks like:



the top two qubits r “visible” (measured to get samples), the bottom two r “ancilla” (traced out to create a mixed state). the key insight: tracing out ancilla qubits from a pure entangled state creates a mixed state on the visible qubits, which can approximate a thermal state.

5 Quantum GANs — The Main Framework

now for the main event. QGANs r arguably the most developed quantum generative model, thanks largely to zoufal et al. 2019.

5.1 QGAN Architecture

Definition 5.1 (Quantum GAN). a quantum GAN consists of:

- quantum generator: a PQC $G(\theta)$ that transforms an initial state (possibly with random input) into a quantum state encoding a probability distribution
- discriminator: either a classical neural network or a quantum circuit $D(\phi)$ that distinguishes real data from generated samples

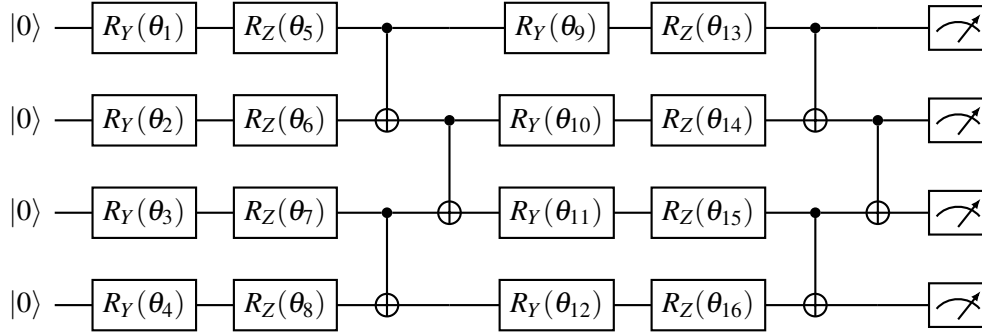
the most common setup (and the one zoufal et al. use):

- quantum generator with classical discriminator
- generator output is measured to produce classical samples
- discriminator is a standard neural network that takes these samples as input

Intuition. the quantum generator is like a quantum computer that outputs random numbers. but its not just any random numbers — the PQC shapes the distribution so that the output statistics match the target distribution. the discriminator’s job is to tell whether samples came from the real distribution or the quantum generator.

5.2 QGAN Generator Circuit

the generator circuit typically has this structure:



this circuit has:

- $n = 4$ qubits, so it can represent distributions over $2^4 = 16$ values
- single-qubit rotation layers (R_Y , R_Z) for expressivity
- CNOT entangling layers for creating correlations
- two “layers” (repetitions of the rotation + entangling pattern)
- measurements in the computational basis to produce samples

the output probability distribution is:

$$p_G(x) = |\langle x | U(\theta) | 0^n \rangle|^2, \quad x \in \{0, 1, \dots, 2^n - 1\}$$

with n qubits, we can represent distributions over 2^n outcomes using only $O(\text{poly}(n))$ parameters. this exponential compression is the key potential advantage.

5.3 Noise Input for QGANs

in classical GANs, the generator takes a random noise vector $\mathbf{z}$ as input. for QGANs, there are several approaches to incorporating randomness:

1. inherent quantum randomness: just measure the output of the PQC. quantum mechanics provides the randomness “for free” through the Born rule. this is the most common approach.
2. random initial rotations: apply random single-qubit rotations before the trainable circuit. the angles are sampled from some distribution each time.
3. ancilla-based: use additional qubits in a fixed superposition state and trace them out.

most QGAN implementations use approach 1 — the quantum randomness from measurement is sufficient.

6 Training QGANs

6.1 The Quantum Minimax Game

the QGAN training objective mirrors the classical GAN:

$$\min_{\theta} \max_{\phi} V(\theta, \phi) = \mathbb{E}_{x \sim p_{\text{real}}} [\log D_{\phi}(x)] + \mathbb{E}_{x \sim p_{G_{\theta}}} [\log(1 - D_{\phi}(x))]$$

the key difference: $p_{G_{\theta}}$ comes from measuring a quantum circuit, so the gradients with respect to θ involve quantum parameter-shift rules (which we covered in lecture 10).

6.2 Alternating Optimization

the training loop:

- Step 1: sample from generator: run the quantum circuit $G(\theta)$ many times, measure to get samples $\{x_{\text{gen}}^{(1)}, \dots, x_{\text{gen}}^{(M)}\}$
- Step 2: update discriminator: using real samples and generated samples, update ϕ to maximize V . this is classical optimization (backprop if D is a neural network).
- Step 3: update generator: using the discriminator's feedback, update θ to minimize V . this uses the parameter-shift rule:

$$\frac{\partial V}{\partial \theta_k} = \frac{V(\theta_k + \pi/2) - V(\theta_k - \pi/2)}{2}$$

- Step 4: repeat until convergence (or until u lose patience — QGAN training can be slow).

Key Point. each evaluation of the generator requires running the quantum circuit many times (shots) to estimate probabilities. with current hardware, this is the bottleneck. u need enough shots for good probability estimates, but each shot takes time on real hardware.

6.3 Loss Functions for QGANs

besides the standard GAN loss (based on JS divergence), several alternatives have been explored:

1. JS divergence (standard GAN):

$$\text{JS}(p_{\text{real}} \| p_G) = \frac{1}{2} \text{KL}(p_{\text{real}} \| m) + \frac{1}{2} \text{KL}(p_G \| m), \quad m = \frac{p_{\text{real}} + p_G}{2}$$

2. Wasserstein distance:

$$W_1(p_{\text{real}}, p_G) = \inf_{\gamma \in \Pi(p_{\text{real}}, p_G)} \mathbb{E}_{(x,y) \sim \gamma} [\|x - y\|]$$

this is particularly good for QGANs bcs it provides smoother gradients even when p_{real} and p_G have non-overlapping support (which happens a lot in early training).

3. KL divergence:

$$\text{KL}(p_{\text{real}} \| p_G) = \sum_x p_{\text{real}}(x) \log \frac{p_{\text{real}}(x)}{p_G(x)}$$

this can blow up when $p_G(x) = 0$ for some x where $p_{\text{real}}(x) > 0$. not ideal for quantum circuits that might have structural zeros.

Example 6.1. suppose the target distribution is a discretized log-normal (relevant for financial applications). with $n = 3$ qubits, we have 8 bins. the generator circuit needs to learn 8 probabilities summing to 1. the Wasserstein loss typically converges faster than JS for this kind of structured distribution.

7 Zoufal et al. 2019 — The Foundational QGAN Paper

this paper (“Quantum Generative Adversarial Networks for Learning and Loading Random Distributions”) is the most important reference for QGANs. lets break it down.

7.1 Key Contributions

1. demonstrated that a quantum generator can learn arbitrary probability distributions
2. showed how to load distributions into quantum states efficiently (this is the killer app)
3. ran experiments on real IBM quantum hardware
4. proposed the specific architecture that most subsequent work builds on

7.2 The Loading Problem

Key Point. why do we care about loading distributions into quantum states? bcs many quantum algorithms (quantum Monte Carlo, quantum amplitude estimation, quantum option pricing) need the input data encoded as quantum amplitudes. classically preparing an arbitrary n -qubit state requires $O(2^n)$ gates. but if a QGAN can learn to generate the distribution with a shallow circuit, thats exponentially faster.

formally: given a target distribution p_{target} over $\{0, 1, \dots, 2^n - 1\}$, we want to find circuit parameters θ^* such that:

$$|\langle x | U(\theta^*) | 0^n \rangle|^2 \approx p_{\text{target}}(x) \quad \forall x$$

this is essentially amplitude encoding, but achieved through learning rather than explicit state preparation.

7.3 Architecture Details

zoufal et al. use:

- quantum generator: hardware-efficient ansatz with R_Y and R_Z rotations + CX entangling gates
- classical discriminator: simple neural network (few layers, sigmoid output)
- loss: standard GAN loss (cross-entropy)
- optimizer: ADAM for the discriminator, quantum natural gradient or ADAM for the generator

their circuit depth scales as $O(n \cdot L)$ where n is the number of qubits and L is the number of layers. they found that $L \sim n$ layers was sufficient for the distributions they tested.

7.4 Results

key results from the paper:

- successfully learned log-normal, triangular, and bimodal distributions on 3 qubits
- KL divergence between learned and target distribution: < 0.1 in most cases
- real hardware experiments on IBM quantum devices showed feasibility despite noise
- training required ~ 2000 iterations for convergence
- the learned circuits were shallow enough for NISQ execution

the practical impact: this gives a way to do efficient quantum state preparation for quantum finance algorithms. instead of using $O(2^n)$ gates to prepare a price distribution, train a QGAN with $O(\text{poly}(n))$ parameters.

8 Born Machines

Born machines r the simplest quantum generative model. they r so clean and elegant that they deserve their own section.

8.1 The Born Rule as a Generative Model

Definition 8.1 (Born Machine). a Born machine is a parameterized quantum circuit $U(\theta)$ that defines a probability distribution through the Born rule:

$$p_\theta(x) = |\langle x | U(\theta) | 0^n \rangle|^2$$

where $x \in \{0, 1\}^n$ is a bitstring outcome.

Intuition. this is literally the simplest thing u can do: apply a quantum circuit and measure. the probabilities u get from measurement define ur generative model. no discriminator, no adversarial training (necessarily), no hidden units. just quantum mechanics doing its thing.

the state before measurement is:

$$|\psi(\theta)\rangle = U(\theta) | 0^n \rangle = \sum_{x \in \{0, 1\}^n} \alpha_x(\theta) | x \rangle$$

so $p_\theta(x) = |\alpha_x(\theta)|^2$. the amplitudes α_x live in a 2^n -dimensional Hilbert space, giving exponential expressiveness with polynomial parameters.

8.2 Training Born Machines

since theres no discriminator, we need a different training signal. the two main approaches:

1. Maximum Mean Discrepancy (MMD):

Definition 8.2 (MMD Loss). the MMD between the Born machine distribution p_θ and the target distribution q is:

$$\text{MMD}^2(p_\theta, q) = \mathbb{E}_{x, x' \sim p} [k(x, x')] - 2\mathbb{E}_{x \sim p, y \sim q} [k(x, y)] + \mathbb{E}_{y, y' \sim q} [k(y, y')]$$

where k is a kernel function (e.g., Gaussian kernel $k(x, y) = \exp(-\|x - y\|^2 / 2\sigma^2)$).

the gradient of the MMD loss can be computed using the parameter-shift rule:

$$\frac{\partial \text{MMD}^2}{\partial \theta_k} = \frac{\text{MMD}^2(\theta_k + \pi/2) - \text{MMD}^2(\theta_k - \pi/2)}{2}$$

2. KL divergence:

$$\text{KL}(q \| p_\theta) = \sum_x q(x) \log \frac{q(x)}{p_\theta(x)}$$

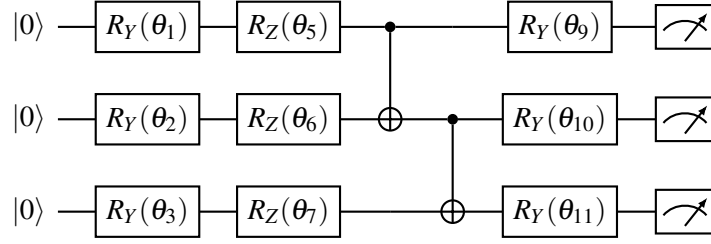
this requires estimating $p_\theta(x)$ from samples, which needs many shots. the gradient is:

$$\frac{\partial \text{KL}}{\partial \theta_k} = - \sum_x q(x) \frac{1}{p_\theta(x)} \frac{\partial p_\theta(x)}{\partial \theta_k}$$

Key Point. MMD is generally preferred for Born machine training bcs (1) it doesnt require density estimation, (2) the kernel provides a smooth comparison between distributions, and (3) it handles zero-probability events gracefully.

8.3 Quantum Circuit Born Machine (QCBM)

the quantum circuit Born machine (liu and wang, 2018) is a specific implementation:



the key property of QCBMs: they can efficiently represent distributions that would require exponentially many parameters classically. specifically, distributions arising from quantum processes (like the output of a quantum circuit) can be modeled by QCBMs with polynomial resources, but might need exponential classical resources.

Theorem 8.1 (Expressiveness of Born Machines). there exist probability distributions that can be sampled by a polynomial-size quantum circuit but cannot be represented by any polynomial-size classical generative model (assuming standard complexity-theoretic conjectures). specifically, distributions related to the permanent of a matrix (aaronson and arhipov, 2011 — boson sampling) r in this class.

9 Applications

9.1 Financial Data Generation

this is probably the most developed application of QGANs. the idea:

1. have historical price data (e.g., stock returns) that follows some distribution
2. train a QGAN to learn this distribution
3. now u have a quantum state encoding the distribution: $\sum_x \sqrt{p(x)} |x\rangle$
4. use this as input to quantum algorithms like quantum amplitude estimation (QAE) for option pricing

the pipeline:

$$\text{classical data} \xrightarrow{\text{QGAN training}} \text{quantum state} \xrightarrow{\text{QAE}} \text{expected payoff}$$

why this matters: quantum amplitude estimation gives a quadratic speedup over classical Monte Carlo for computing expectations. but u need the distribution loaded into a quantum state first. the QGAN solves the loading problem.

Example 9.1. option pricing: u want to compute $\mathbb{E}[f(S_T)]$ where S_T is the stock price at maturity and f is the payoff function. classically, Monte Carlo converges as $O(1/\sqrt{N})$. QAE gives $O(1/N)$ — quadratic speedup. but u need $|\psi\rangle = \sum_x \sqrt{p(x)} |x\rangle$ where $p(x)$ approximates the price distribution. the QGAN gives u this state with a shallow circuit.

9.2 Drug Molecule Generation

another promising application: generating novel molecular structures.

the idea: represent molecules as graphs or SMILES strings, discretize them, and train a quantum generative model to produce new valid molecules with desired properties.

challenges:

- molecules have complex structural constraints (valence rules, stability)
- the discrete structure makes training harder
- it's not clear that quantum generators have an advantage for this specific task

but there are reasons for optimism: molecular properties are governed by quantum mechanics, so a quantum model might naturally capture the right correlations. this is speculative but intriguing.

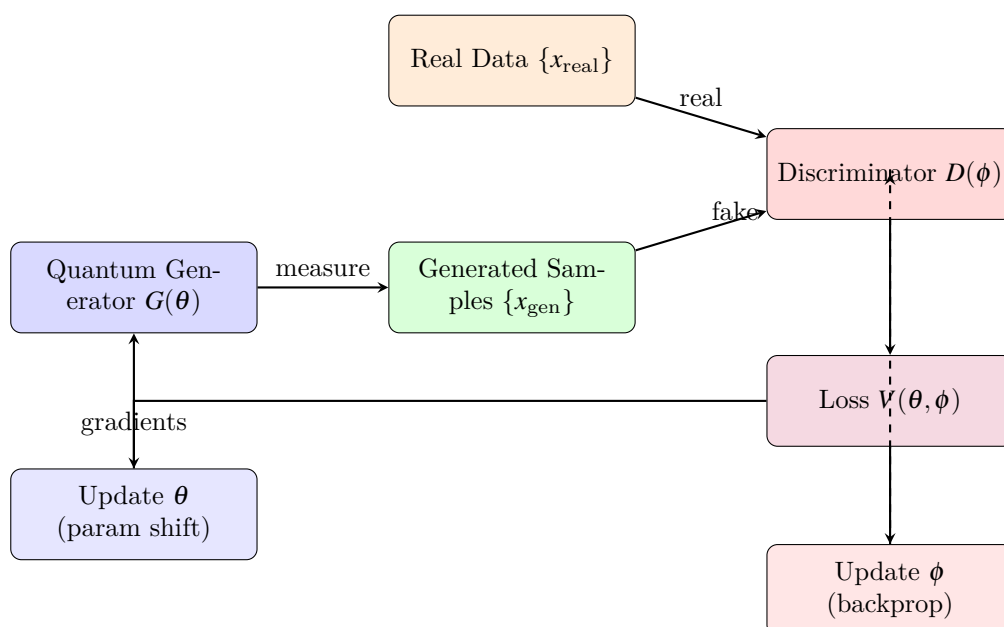
9.3 Anomaly Detection

train a quantum generative model on normal data. then, new data that the model assigns low probability to is flagged as anomalous. this has been explored for:

- credit card fraud detection
- network intrusion detection
- particle physics (rare event detection at the LHC)

10 QGAN Training Loop — Visual

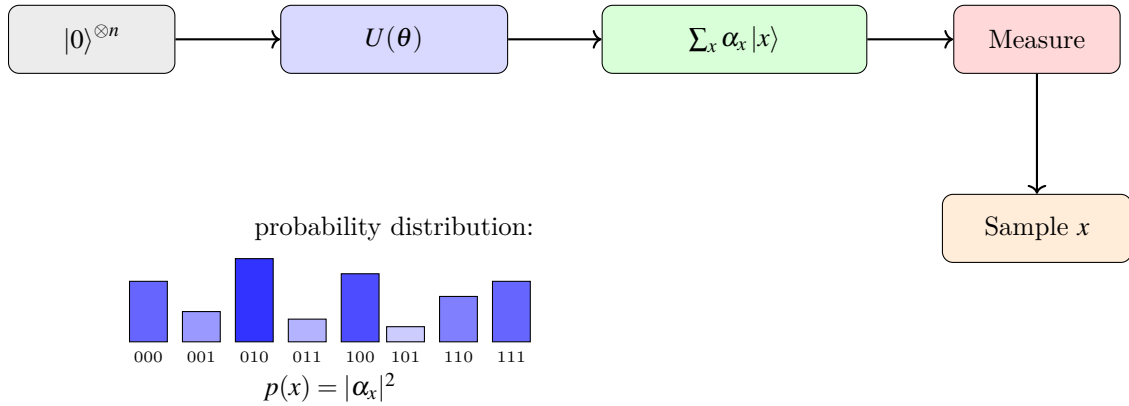
here's the full picture of how QGAN training works:



the dashed arrow shows the discriminator update feeding back into itself. the solid arrows show the main flow: generate samples, discriminate, compute loss, update both networks.

11 Born Machine Sampling Concept

lets visualize how a Born machine generates samples:



the circuit transforms $|0\rangle^{\otimes n}$ into a superposition, and measurement collapses it to a specific bitstring with probability given by the Born rule. by training the circuit parameters, we shape the probability distribution to match our target.

12 Mode Collapse and Training Challenges

training quantum generative models is hard. lets be honest about the challenges.

12.1 Mode Collapse

just like classical GANs, QGANs can suffer from mode collapse:

Definition 12.1 (Mode Collapse). mode collapse occurs when the generator learns to produce only a subset of the modes (peaks) in the target distribution, ignoring the rest. the generator gets “stuck” producing a few high-probability outcomes.

in the quantum case, mode collapse means the circuit state has high amplitude on only a few computational basis states, even though the target distribution is more spread out.

causes in the quantum setting:

- limited circuit depth (not enough entanglement to represent multimodal distributions)
- barren plateaus making it hard to escape local minima
- discriminator overpowering the generator

12.2 Shot Noise

Key Point. this is the uniquely quantum challenge: every probability estimate from a quantum circuit has statistical uncertainty from finite sampling (shot noise). if u use S shots, the uncertainty in probability estimates is $O(1/\sqrt{S})$.

this affects training in several ways:

- gradients r noisy: the parameter-shift rule requires estimating expectation values, each with shot noise

- the discriminator receives noisy samples, making its gradients noisy too
- for Wasserstein QGANs, the critic function must be estimated from noisy samples

the shot noise compounds with the usual stochastic gradient noise. in practice, u need $O(10^3 - 10^4)$ shots per circuit evaluation, and the parameter-shift rule doubles that (two circuit evaluations per gradient component).

total cost per training step: $O(P \cdot S)$ circuit evaluations, where P is the number of parameters and S is the number of shots. this adds up fast.

12.3 Hardware Noise

on real NISQ devices, gate errors, decoherence, and readout errors all distort the output distribution. this means the generator is effectively learning a noisy version of the target distribution, convolved with the hardware noise.

mitigation strategies:

- error mitigation techniques (zero-noise extrapolation, probabilistic error cancellation)
- noise-aware training (include a noise model in the optimization loop)
- using more robust loss functions (Wasserstein distance is more tolerant of noise than KL divergence)

12.4 Barren Plateaus (Again)

yes, barren plateaus strike again. for quantum generative models:

- random initialization of deep circuits leads to exponentially vanishing gradients
- the problem is worse for global loss functions (like fidelity between full quantum states)
- local loss functions (like MMD with local kernels) r more resistant

the standard mitigations apply: shallow circuits, structured initialization, local cost functions.

13 Comparison: QBM vs QGAN vs Born Machine

lets put it all together:

Property	QBM	QGAN	Born Machine
model state	thermal state $e^{-\beta H}/Z$	pure state from PQC	pure state from PQC
output type	mixed state (density matrix)	classical samples (after measurement)	classical samples (after measurement)
training signal	quantum relative entropy	adversarial (discriminator)	MMD or KL divergence
parameters	Hamiltonian couplings J_{ij}, h_i, Γ_i	circuit rotations θ_k	circuit rotations θ_k
can represent mixed states	yes (naturally)	only via ancilla tracing	only via ancilla tracing
training difficulty	hard (need thermal state prep)	moderate (adversarial instability)	easiest (direct optimization)
key paper	amin et al. 2018	zoufal et al. 2019	liu and wang 2018
best for	quantum data, thermal distributions	loading classical distributions	simple generative tasks

Intuition. if u want the simplest quantum generative model, use a Born machine. if u want to load a classical distribution into a quantum state (e.g., for quantum finance), use a QGAN. if u r working with genuinely quantum data (e.g., thermal states of physical systems), consider a QBM.

14 Advanced Topics

14.1 Fully Quantum GANs

in a fully quantum GAN, both the generator and discriminator r quantum circuits. the discriminator takes a quantum state as input (no intermediate measurement) and outputs a classification.

the loss function becomes:

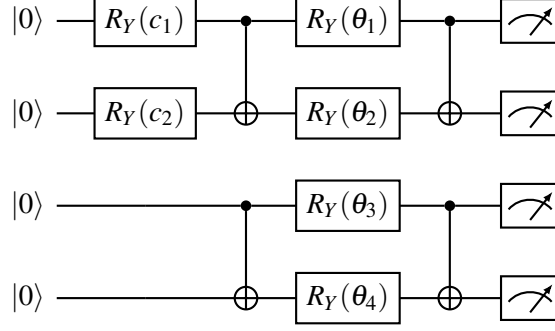
$$V = \text{Tr}(D \cdot \rho_{\text{real}}) - \text{Tr}(D \cdot \rho_{\text{gen}})$$

where D is a quantum observable (the “discriminator operator”).

lloyd and weedbrook (2018) first proposed this. the advantage: the discriminator can detect quantum features (coherences, entanglement) that would be invisible to a classical discriminator working with measurement outcomes.

14.2 Conditional QGANs

just like classical conditional GANs, u can condition the quantum generator on a label or class:



here c_1, c_2 are determined by the conditioning variable (e.g., class label). the circuit generates samples from the class-conditional distribution.

14.3 Quantum Wasserstein GAN

chakrabarti et al. (2019) proposed a quantum version of WGAN where the critic is a quantum circuit. the Wasserstein distance between quantum states is:

$$W(\rho, \sigma) = \max_{\|O\|_L \leq 1} |\text{Tr}(O(\rho - \sigma))|$$

where $\|O\|_L \leq 1$ means the observable O has bounded Lipschitz norm. in practice, this is approximated by parameterizing O as a PQC and adding a gradient penalty.

15 Practical Considerations

15.1 How Many Qubits Do U Need?

for a QGAN loading a distribution over K values:

$$n = \lceil \log_2 K \rceil \text{ qubits}$$

so for a distribution over 256 values (e.g., 8-bit discretization), u need 8 qubits. for 1024 values, 10 qubits. this is modest by quantum computing standards, but remember that each additional qubit doubles the Hilbert space dimension.

15.2 Circuit Depth vs Expressivity

Theorem 15.1 (Approximate Universality). a hardware-efficient ansatz with L layers of single-qubit rotations and entangling gates on n qubits can approximate any n -qubit state to accuracy ϵ with:

$$L = O\left(\frac{2^n}{n}\right)$$

layers in the worst case. but for structured distributions (which is what we typically care about), much fewer layers suffice.

in practice, zoufal et al. found that $L \sim 3n$ layers worked well for distributions on up to 5 qubits.

15.3 Comparison with Classical Loading

classical state preparation of an arbitrary n -qubit state requires $O(2^n)$ CNOT gates. the QGAN approach requires:

- training time: $O(T \cdot P \cdot S)$ circuit evaluations (T iterations, P parameters, S shots)
- inference time: $O(n \cdot L)$ gates per sample

if $n \cdot L \ll 2^n$ (which it is for reasonable n and L), the QGAN approach wins at inference time. the training is expensive but is a one-time cost.

16 Summary

ok lets wrap up wt we covered:

1. classical foundations: Boltzmann machines use energy-based models with contrastive divergence training. GANs use adversarial training with a minimax game.
2. quantum Boltzmann machines: replace classical energy with quantum Hamiltonian. transverse field terms create quantum coherences that can represent correlations beyond classical BMs. training requires thermal state preparation, which is hard.
3. QGANs: quantum generator (PQC) + classical or quantum discriminator. the main application is loading probability distributions into quantum states. zoufal et al. 2019 showed this works on real hardware.
4. Born machines: the simplest quantum generative model. probability from the Born rule. trained with MMD or KL divergence. can represent exponentially complex distributions with polynomial parameters.
5. applications: financial data loading (for quantum Monte Carlo), drug molecule generation, anomaly detection.
6. challenges: mode collapse, shot noise, hardware noise, barren plateaus. training quantum generative models is harder than training quantum classifiers.

the big picture: quantum generative models exploit the exponential dimensionality of Hilbert space to represent complex distributions compactly. the most practical near-term application is distribution loading for quantum algorithms. the field is young and theres a lot of room for new ideas.

next lecture: we step back and ask the big questions — wt is the actual power of QML? where is quantum advantage proven, and where is it just hoped for? we'll also wrap up the course.

17 Further Reading

- [1] zoufal, lucchi, woerner. “quantum generative adversarial networks for learning and loading random distributions.” npj Quantum Information, 2019.
- [2] liu, wang. “differentiable learning of quantum circuit Born machines.” Physical Review A, 2018.

- [3] amin, andriyash, rolfe, kulchytskyy, melko. “quantum Boltzmann machine.” Physical Review X, 2018.
- [4] lloyd, weedbrook. “quantum generative adversarial learning.” Physical Review Letters, 2018.
- [5] chakrabarti, huang, musser, et al. “quantum Wasserstein generative adversarial networks.” NeurIPS, 2019.
- [6] goodfellow et al. “generative adversarial nets.” NeurIPS, 2014.
- [7] hinton. “training products of experts by minimizing contrastive divergence.” Neural Computation, 2002.